

מבני נתונים – פרויקט מספר 1 – עץ מאוזן

שמות המגשים:
pustylnik, 213137367, איריס פוסטילניק,
kseniay, 345156905, קסניה ירמנקו,

תיעוד חיצוני

תיאור המחלקה AVLNode:

המחלקה מתארת צומת בעץ AVL. למחלקה יש את השדות הבאים:
key – המפתח של הצומת.
value – הערך של הצומת.
left, right – בן שמאלי/ימני של הצומת.
parent – אב הצומת.
height – גובה הצומת.

במחלקה יש בנאי המתחל רק את השדות key, value בערכים שמקבל כקלט, ואת שאר השדות מאתחל באופן טבעי.
בנוסף, ישנה מתודה is_real_node המחזירה האם הצומת הוא צומת אמיתי ולא וירטואלי.

תיאור המחלקה AVLTree:

המחלקה מתארת עץ AVL. למחלקה יש את השדות הבאים:
sizeOfTree – גודל העץ, כלומר כמות הצמתים.
root – שורש העץ.
exLeaf – עלה חיצוני.

במחלקה יש את הבנאי המתחל את גודל העץ ל-0, את השורש לnone ואת העלה החיצוני לעלה וירטואלי.

מתודות:

`new_node(self, key, value)` – מתודת עזר הרצה בזמן $O(1)$, נועדה לצורך יצירת צומת חדשה עם גובה אפס.
`search(self, key)` – מתודת החיפוש שהתבקשנו לממש. קוראת למתודת העזר `down_search(self.root, key)` בלבד. הסיבוכיות הינה כסיבוכיות של `down_search`.
`down_search(self, currNode, key)` – מתודת עזר שמבצעת חיפוש קלאסי בתת עץ ששורשו זה הצומת הנתון. במתודה ישנה לולאת while והיא מתבצעת לכל היותר $\log n$ פעמים ולכן סיבוכיות המתודה הינה $O(\log n)$.
`finger_search(self, key)` – מתודת חיפוש בהינתן מצביע למקסימלי שהתבקשנו לממש. הפונק' קוראת למתודה `max_node` שמחזירה צומת עם מפתח מקסימלי, עולים בעץ עד המפתח הראשון שקטן מהמפתח הנתון, ואז יורדים בחיפוש בעזרת מתודת העזר `down_search`. לכן הסיבוכיות הינה $O(\log n)$.
`max_node(self)` – מתודה המחזירה את הצומת עם המפתח המקסימלי שהתבקשנו לממש. מאחר ויש בה לולאת while המתבצעת לכל היותר $\log n$ פעמים, סיבוכיות המתודה הינה $O(\log n)$.
`insert(self, key, val)` – מתודה המכניסה איבר לעץ שהתבקשנו לממש. המתודה קוראת למתודת העזר `searching_for_insert(x, key)` המחזירה את האב של הצומת החדש ו-h אורך המסלול, סיבוכיותה הינה $O(\log n)$. בנוסף קוראת ל-`new_node(key, val)` שסיבוכיותה $O(1)$, וגם ל-`inserting_node(y, newNode)` שזו מתודת עזר להכנסת צומת שרצה בזמן $O(1)$, וגם למתודה `update_height` שרצה בזמן $O(\log n)$. לבסוף קוראת למתודת העזר `balance_AVLTree(y, 1, promoteCases)` שמבצעת איזון לעץ, סיבוכיותה $O(\log n)$. לכן, בסה"כ סיבוכיות הריצה של `insert` הינה $O(\log n)$.
`finger_insert(self, key, val)` – מתודה המכניסה איבר בהינתן מצביע למקסימלי שהתבקשנו לממש. מתודה קוראת למתודות העזר: `max_node` שרצה בזמן $O(\log n)$, `new_node(key, val)` שרצה בזמן $O(1)$, כעת או שנבצע `inserting_node(x, newNode)` שרצה בזמן $O(1)$, או שנכנס ל-while שעולה מהמקסימום לצומת הראשון הקטן מ-k הנתון, מקסימום $\log n$ חזרות, שאחריה נפעיל את `searching_for_insert(x, key)` שסיבוכיותה $O(\log n)$ ושוב נבצע הנכסה בעזרת `inserting_node` שעולה לנו $O(1)$, וגם למתודה `update_height` שרצה בזמן $O(\log n)$. לבסוף, נבצע איזון בעזרת `balance_AVLTree` שעולה לנו $O(\log n)$. סה"כ סיבוכיות `finger_search` הינה $O(\log n)$.

`searching_for_insert(x, key)` - מתודת עזר המחזירה את האב של הצומת החדש ו-h אורך המסלול, סיבוכיותה הינה $O(\log n)$, מאחר ומבצעת לולאת `while` לכל היותר $\log n$ פעמים.
`inserting_node(y, newNode)` - מתודת עזר להכנסת צומת שרצה בזמן $O(1)$.
`update_height(self, node)` - מתודה המעדכנת את הגובה של `node`. מאחר וזה רק חישובים הסיבוכיות הינה $O(1)$.
`balance_AVLTree(self, node, dtype, promoteCases)` - מתודת עזר שנועדה לבצע את תהליך האיזון במקרה של הכנסה ובמקרה של מחיקה. ה-`dtype` מתאר לנו איזה סוג של איזון צריך, הכנסה או מחיקה. ה-`node` זה המקום ממנו אנחנו מתחילים את תהליך האיזון. המתודה מבצעת שימוש במתודות העזר `balance_factor(node)` שמחזירה את ה-`balance factor` של הצומת, שסיבוכיותה $O(1)$, בנוסף מתמשת במתודות העזר `get_height` בשביל לקבל את הגבהים של הילדים של הצומת הנתון. סה"כ המתודה מבצעת חלוקה למקרים כפי שנלמד בהרצאה, ובכל אחד מהמקרים קוראת למתודות העזר `rotation` שמטרתה לבצע את הגלגולים הנדרשים בתהליך האיזון שסיבוכיותה $O(1)$. לכן, סה"כ הסיבוכיות של מתודת האיזון הינה $O(\log n)$.
`balance_factor(self, node)` - מתודת עזר המחשבת את ה-`balance factor` של `node`, רצה בזמן $O(1)$.
`get_height(self, node)` - מתודת עזר המחזירה גובה של `node`, רצה בזמן $O(1)$.
`rotation(self, nodeB, dir)` - מתודת עזר המבצעת את הגלגול הדרוש, בהתאם `dir` שהתבקש, כפי שנלמד בהרצאה. מתשמשת במתודות העזר `update_height` שרצה בזמן $O(1)$, ולכן זו גם הסיבוכיות של `rotation`.
`delete(self, node)` - מתודת המחיקה שהתבקשנו לממש. המתודה משתמשת במתודות העזר `successor` שרצה בזמן $O(\log n)$, המתודות עובדות לפי הנלמד בהרצאה ומטפלות בשלושת המקרים האם יש לנו שני בנים, בן אחד או בכלל אין. לבסוף מבצעת איזון בעזרת מתודת העזר `balance_AVLTree` שעולה לנו $O(\log n)$. לכן סה"כ המתודה בעל סיבוכיות של $O(\log n)$.
`successor(self, node)` - מתודת עזר המחזירה את ה-`successor` של `node`, כפי שנלמד בהרצאה. המתודה משתמשת במתודות העזר `minNode` שעולה לנו $O(\log n)$, ולאחר מכן יש לה לולאת `while` שמתבצעת לכל היותר $\log n$ פעמים, לכן הסיבוכיות הינה $O(\log n)$.
`minNode(self, node)` - מתודת עזר המחשבת את ה-`node` המינימלי בעץ. מאחר ויש לה לולאת `while` שמתבצעת לכל היותר $\log n$ פעמים הסיבוכיות הינה $O(\log n)$.
`join(self, tree2, key, val)` - מתודת האיחוד שהתבקשנו לממש. הפונק' מאחדת את שני העצים, הנתון כקלט `self` ו-`tree2` או `self`, על גבי הקלט `key` או `val`, לאחר בדיקה מי העץ עם הגובה הגדול יותר ובלסוף אנו משנים את `self` להיות העץ המעודכן לאחר פעולת האיחוד. היא משתמשת במתודות העזר: `get_height` רצה בזמן $O(1)$, `insert` שהתבקשנו לממש שרצה בזמן $O(\log n)$, ומשתמשת במתודות העזר `update_height` שרצה בזמן $O(1)$, ולבסוף מאזנת את העץ בעזרת מתודת העזר `balance_AVLTree` שעולה לנו $O(\log n)$. סה"כ העלות הינה $O(\log n)$.
`split(self, node)` - מתודת החצייה שהתבקשנו לממש. היא מפרקת את העץ הנוכחי לשני עצים כך שעץ אחד עם איברים גדולים מ-`node` ועץ שני עם איברים הקטנים מ-`node`. המתודה משתמשת במתודות העזר הבאות: `join`, שעולה לנו $O(\log n)$, `detach_node` שחותכת את החיבורים של `node` לאביו וילדיו שעולה $O(1)$. במתודה יש לנו לולאת `while` שרצה לכל היותר $\log n$ פעמים, ובאה מתבצעת קריאה ל-`join`, לכן הסיבוכיות הינה $O((\log n)^2)$.
`detach_node(self, node)` - מתודת עזר המוחקת את הקשרים שבין ה-`node` לבין אביו וילדיו, היא רצה בזמן של $O(1)$.
`avl_to_array(self)` - מתודה שמציגה את העץ הנוכחי למערך שהתבקשנו לממש. הפונק' קוראת למתודות העזר `in_order` שסיבוכיותה $O(n)$ ולכן זו גם הסיבוכיות של המתודה שלנו.
`in_order(self, currNode, avlArray)` - מתודת עזר רקורסיבית שמבצעת `in order` כפי שנלמד בהרצאה. סיבוכיותה הינה $O(m)$ כאשר m זה מס הצמתים בתת העץ של `currNode`.
`size(self)` - מתודה המחזירה את גודל העץ שהתבקשנו לממש. מאחר והכנסו אותו כאחד מהשדות שמגדירים את הטיפוס של עץ `avl` זהו פשוט `getter` ולכן הסיבוכיות הינה $O(1)$.
`get_root(self)` - מתודה המחזירה את השורש של העץ שהתבקשנו לממש, זהו פשוט `getter` לשדה `root`, ולכן הסיבוכיות הינה $O(1)$.
`update_height(self, node)` - מתודה המעדכנת את הגובה של `node`. מאחר וזה רק חישובים הסיבוכיות הינה $O(1)$.

חלק ניסויי/תאורטי

בשאלה זו נדון ב insertion-sort באמצעות AVL Finger Tree. המיון מתבצע באופן הבא: מכניסים את האיברים לפי הסדר (הלא ממין) אל העץ, כאשר החיפוש בהכנסת כל איבר חדש מתחיל מהמקסימום הנוכחי, ובסיום מבצעים סריקת in-order לקבלת הסדר הממוין. עבור עלות בניית העץ, ננתח בנפרד את עלות החיפוש ואת מספר פעולות האיזון.

- לצורך הניתוח, נמין מערכים בגדלים שונים. גודל המערך שנמין יהיה $n = 300 \cdot 2^i$ כאשר $i = 1, \dots, 10$, ואיבריו יהיו הטבעיים עד n . למשל, עבור $i = 1$ המערך בגודל 600, ועבור $i = 5$ המערך בגודל 9600.
- לכל גודל n של מערך, נבצע 4 ניסויים נפרדים:
 - בניסוי הראשון נמין מערך ממין, מקטן לגדול.
 - בניסוי השני נמין מערך ממין הפוך, מגדול לקטן.
 - בניסוי השלישי סדר האיברים במערך יהיה אקראי.
 - בניסוי הרביעי ניקח מערך ממין ועבור כל אינדקס פרט לאחרון $i = 0, \dots, n-2$, נבצע החלפה עם האיבר הבא $A[i] \leftrightarrow A[i+1]$ בסיכו חצי (שימו לב שייתכן שאיבר יוחלף מספר פעמים).

הערה: בסעיפים הבאים, עבור ניסויים אקראיים, יש לקחת את הממוצע על פני 20 ניסויים.

- יש למלא בטבלה הבאה את סך עלויות האיזון **ללא גלגולים** עבור כל אחד מהניסויים. הסבירו מהו החסם העליון התאורטי על סך עלויות האיזון כולל גלגולים, והאם הערכים בטבלה מתאימים. לסיום, נמקו מדוע תוספת הגלגולים לספירה אינה משנה אסימפטוטית.
- פתרון:** החסם העליון התאורטי על סך עלויות האיזון, כולל הגלגולים הינו: $O(n \log n)$. זאת מכיוון שעלות הכנסה בודדת הינה $O(\log n)$, כפי שראינו בהרצאה. מאחר ואנו מכניסים n איברים סה"כ $O(n)$. לכן העלות הכוללת הינה $O(n \log n)$. נשים לב שהערכים בטבלה מתאימים לחסם הזה, מכיוון שזהו חסם עליון, אין זה אומר כי הם צריכים להיות בדיוק שווים לכך. נשים לב שאסימפטוטית תוספת הגלגולים לספירה אינה משנה מאחר והיא עולה לנו $O(1)$.

להלן הטבלה:

מספר סידורי i	עלות איזון במערך ממין	עלות איזון במערך ממין - הפוך	עלות איזון במערך מסודר אקראית	עלות איזון במערך עם היפוכים סמוכים אקראיים
1	590	590	274.5	502.1
2	1,189.00	1,189.00	549	1,013.40
3	2,388.00	2,388.00	1,118.50	2,043.90
4	4,787.00	4,787.00	2,236.70	4,091.60
5	9,586.00	9,586.00	4,472.60	8,201.70
6	19,185.00	19,185.00	8,920.50	16,423.70
7	38,384.00	38,384.00	17,883.20	32,863.30
8	76,783.00	76,783.00	35,802.80	65,735.90
9	153,582.00	153,582.00	71,568.40	131,478.20
10	307,181.00	307,181.00	143,104.40	262,893.20

- בהינתן מערך A בגודל n נגדיר היפוך בתור זוג אינדקסים $i < j$ כך שמתקיים $A[i] > A[j]$, ונסמן את מספר ההיפוכים הכולל במערך ב- I . ניתן לשים לב כי באופן כללי $0 \leq I \leq \binom{n}{2}$ וככל שיש פחות היפוכים כך המערך קרוב יותר לממוין. יש למלא בטבלה הבאה את מספר ההיפוכים במערך הקלט עבור כל אחד מהניסויים (מספיק עד $i = 5$).

פתרון: להלן הטבלה:

מספר סידורי i	מספר היפוכים במערך ממין	מספר היפוכים במערך ממין - הפוך	מספר היפוכים במערך מסודר אקראית	מספר היפוכים במערך עם היפוכים סמוכים אקראיים
1	0	179700	89750.6	298.2
2	0.00	719,400.00	356210.5	594.80
3	0.00	2,878,800.00	1,441,809.90	1,201.80
4	0.00	11,517,600.00	5,773,645.90	2,405.30
5	0.00	46,075,200.00	23,017,990.00	4,795.40

3. יש למלא בטבלה הבאה את סך עלויות החיפוש עבור כל אחד מהניסויים.
פתרון: להלן הטבלה:

מספר סידורי i	עלות חיפוש במערך ממוין	עלות חיפוש במערך ממוין - הפוך	עלות חיפוש במערך מסודר אקראית	עלות חיפוש במערך עם היפוכים סמוכים אקראיים
1	599	9012	8175.1	1086
2	1,199.00	20,420.00	19001.3	2,178.20
3	2,399.00	45,636.00	42,715.30	4,340.50
4	4,799.00	100,868.00	94,742.80	8,699.60
5	9,599.00	220,932.00	208,890.20	17,399.80
6	19,199.00	480,260.00	459,289.90	34,783.70
7	38,399.00	1,037,316.00	987,591.50	69,579.20
8	76,799.00	2,228,228.00	2,137,629.00	139,187.50
9	153,599.00	4,763,652.00	4,593,260.90	278,339.20
10	307,199.00	10,141,700.00	9,869,697.30	556,838.40

4. כדי לחסום מלמעלה את סך עלויות החיפוש באופן תאורטי, נסמן ב- d_i את מספר האיברים לפני האיבר באינדקס i שגדולים ממנו.
- הסבירו מדוע $I = \sum d_i$.
 - חסמו את עלות החיפוש בעת הכנסת האיבר ה- i כפונקציה של d_i , והסיקו כי סך עלויות החיפוש לאורך סדרת ההכנסות הוא $O(\log \prod_{i=1}^n (d_i + 2))$.
הערה: כאשר הסיבוכיות היא $O(\max(1, \log d))$ עבור $d \geq 0$, ניתן להמיר לביטוי $O(\log(d + 2))$ כדי לעבור לביטוי אחד פשוט וחוקי שחוסם את שניהם מלמעלה.
 - השתמשו באי-שוויון הממוצעים כדי לחסום מלמעלה את סך עלויות החיפוש כפונקציה של n, I . הסבירו מדוע זוהי העלות הדומיננטית מבחינה אסימפטוטית.
 - השוו בין החסם שהתקבל ותוצאות הניסויים.

פתרון:

- השוויון מתקיים מאחר ו- d_i סופר את מס' ההיפוכים שהאיבר i צריך לעשות בכדי שיהיה במקום "הנכון" במערך, כלומר במקום המתאים מבחינת יחס הסדר של המערך. הסכום שלהם זה מס' ההיפוכים שכל אחד מהאיברים צריך לעשות כדי שלבסוף יצא לנו מערך ממוין. נשים לב, שאין פה ספירה כפולה, כי אנחנו כל פעם מסתכלים על איבר חדש, שבעבורו יש איברים אחרים שיש לשנות. לכן, לפי ההגדרה של $I = \sum d_i$.
- נשים לב שעלות החיפוש בחיפוש רגיל בעץ AVL עם n איברים היא תמיד $O(\log n)$, לא משנה איפה האיבר נמצא. לעומת זאת ב-finger search אנחנו מתחילים את החיפוש מאיבר ספציפי ולא מהשורש. לכן אם המרחק שזה מס' האיברים ברצף הממוין, בין ה-finger לבין איבר המטרה הוא k אז עלות החיפוש היא: $O(\log k)$. נשים לב שבעזרת הגדרת d_i , כאשר מכניסים את האיבר ה- i אז העץ כבר מכיל $i-1$ איברים ממוינים, ו- d_i הזה הוא בדיוק המרחק ה- k שציינו למעלה, זה בדיוק המרחק בין סוף העץ לבין המקום החדש של האיבר. לכן עלות הכנסת האיבר ה- i באמצעות finger search הינה $O(\log d_i)$. מאחר ואנו גם מתחשבים במקרה של הכנסת האיבר הראשון שעולה לנו $O(1)$, נקבל כי עלות ההכנסה הכוללת, בעזרת ההערה שצוינה בתרגיל, הינה: $O(\log(d_i + 2))$. העלות הכוללת הינה סכום העלויות לכל איבר ולכן:

$$Total\ cost = \sum_{i=1}^n O(\log(d_i + 2)) = O(\log(\prod_{i=1}^n (d_i + 2)))$$

כאשר השוויון האחרון נובע מחוקי לוגריתמים: $\log a + \log b = \log(ab)$.

III. נרצה למצוא חסם עליון לביטוי מסעיף קודם התלוי רק ב- I, n . לשם כך נרצה למקסם את המכפלה שבתוך ה- $\log$ תחת אילוץ של סכום קבוע. לשם כך נשתמש בא"ש הממוצעים:

$$\sqrt[n]{\prod_{i=1}^n (d_i + 2)} \leq \frac{1}{n} \left(\sum_{i=1}^n (d_i + 2) \right) = \frac{1}{n} \left(\sum_{i=1}^n d_i + \sum_{i=1}^n 2 \right) = \frac{1}{n} (I + 2n)$$

כאשר השוויון האמצעי מתקיים כי הוכחנו $I = \sum d_i$. נעלה את שני האגפים בחזקת n כדי להיפטר מהשורש:

$$\prod_{i=1}^n (d_i + 2) \leq \left(\frac{1}{n} (I + 2n) \right)^n = \left(\frac{I}{n} + 2 \right)^n$$

נחזיר $\log$ ונקבל את עלות החיפוש הכוללת:

$$Cost \leq O \left(\log \left(\left(\frac{I}{n} + 2 \right)^n \right) \right) = O \left(n \log \left(\frac{I}{n} + 2 \right) \right)$$

מבחינה אסימפטוטית למה זו העלות הדומיננטית, נשים לב שעלות האיון והסיבובים חסומה מבחינה amortized על ידי עלות החיפוש. מאחר והחסם מהווה לחסם תחתון תיאורטי למיון מבוסס השוואות כפונק' של I היפוכים, לא ניתן לבצע את המיון במספר פעולות נמוך יותר באופן מהותי, ולכן רכיב החיפוש הוא זה שמכתיב את הסיבוכיות הכוללת. ולכן כלל שמהערך "מסודר" יותר, כלומר I קטן יותר, אז העלות יורדת לכיוון של $O(n)$ כפי שניתן לראות בעזרת החסם שמצאנו. ■

IV. נעבור על כל מקרה לגופו:

○ המקרה הממוין- נשים לב כי כעת $I = 0$. לכן החסם הופך ל- $n \log(2) \approx O(n)$, לכן זוהי העלות המינימלית האפשרית. כעת נביט בטבלה. בשורה 10 עלות החיפוש היא 307199, זהה הערך הנמוך ביותר באותה שורה.

○ המקרה ממוין-הפוך נשים לב ש $I = \frac{n(n-1)}{2}$ ואז $\frac{I}{n} + 2 = \frac{n-1}{2} + 2 = \frac{n+3}{2}$ כלומר $B(n) = n \log \frac{n+3}{2} = \theta(n \log n)$ כלומר העלות תהיה מסדר גודל $n \log n$ ובפרט תהיה הגדולה ביותר מבין המקרים. אכן, לפי הטבלה ניתן לראות שעבור כל i היחס הזה נשמר. ולדוגמא ניקח $i=1$ אז: $B(n) = 600 * 2.48 = 1488$ ולכן $i=179700, n=600$ שזה תואם סדר גודל של תוצאה בטבלה 3 עמוד 1.

○ עבור מערך מסודר אקראית מספר ההיפוכים הצפוי הוא בערך חצי מהמקסימום, כלומר, גם במקרה האקראי החסם העליון של עלות החיפוש הכוללת מסדר גודל $n \log n$, בדומה למקרה הממוין-הפוך, אך לרוב עם קבוע קטן יותר (כי I קטן בערך פי 2 מהמקסימום).

○ עבור מערך "כמעט ממוין": פה מתחילים ממערך ממוין ואז מחליפים שכנים בהסתברות $\frac{1}{2}$. מכיוון שזה לא פרמוטציות כלליות מספר ההיפוכים גודל יותר לאט ובפועל קצב גידול הוא לינארי. אז $B(n) = n \log(2 + \theta(1)) = \theta(n)$ $\Rightarrow \frac{I}{n} = \theta(1) \Rightarrow I = \theta(n)$ ואכן, ניתן לראות בטבלה קצב גידול לינארי יחסית לח בעמודה של היפוכים סמוכים אקראיים והערכים נמוכים בהרבה ממקרים אחרים. לדוגמה ב- $i=10$ עלות החיפוש היא 556,838.40 שהיא מאט מעל הממוין (307,199) והרבה מתחת לאקראי (9,869,697.30) ולממוין-הפוך (10,141,700) בדיוק לפי המסקנה שעבור $I = \theta(n)$ מתקיים שעלות היא $\theta(n)$ ועבור $I = \theta(n^2)$ מתקיים שעלות החיפוש היא $\theta(n \log n)$. ■